WAC
Windows Artefacts Collector

CENTRE D'EXCELLENCE CYBERDEFENSE AEROSPATIALE

Created by the Aerospace Cyber Defence Centre of Excellence (CEC)

Documentation
WAC (Windows Artefacts Collector)

Windows artefact collection software for digital forensic investigation



SYSTEM ARTEFACTS

USER FILES

DIGITAL EVIDENCE

A TOOL IN SUPPORT OF DIGITAL INVESTIGATION

WAC — Windows Artefacts Collector

User manual

Software version	1.4.0
Document edition	September 2026
Publisher	Aerospace Cyber Defence Centre of Excellence (CEC)
	French Air and Space Force Academy, Air Base 701, Salon-de-Provence

This document describes how to use WAC: preparing and running a collection, reading and verifying its results, and the traces it leaves on the examined machine. The figures quoted were measured on a Windows 11 test virtual machine; they are orders of magnitude, not commitments.

Contents

1 Overview **4**

1.1 What WAC is for . . . 4

1.2 The guiding principle: a minimal footprint . . . 4

1.3 What WAC does not do . . . 4

2 How it works **5**

2.1 Reading the disk directly . . . 5

2.2 Exhibit store and working copy . . . 5

2.3 Registry hives and transaction logs . . . 6

2.4 Two-pass extraction . . . 6

2.5 What is still read live . . . 6

3 Preparing a collection **7**

3.1 Requirements . . . 7

3.2 The collection medium . . . 7

3.3 Before going on site . . . 8

4 Running a collection **9**

4.1 Command line . . . 9

4.2 Configuration file: `wac.yml` . . . 9

4.3 Options . . . 11

4.4 Examples . . . 12

4.5 The `--binary` option . . . 13

4.6 What the screen shows . . . 14

4.7 Duration . . . 15

5 Collect here, convert elsewhere **16**

5.1 Why separate . . . 16

5.2 Which mode . . . 17

5.3 The two commands . . . 17

5.4 What `--collect` takes . . . 17

5.5 What `--convert` does . . . 18
5.6 Limitations . . . 19

6 The results **20**
6.1 Layout . . . 20
6.2 The exhibit store manifest . . . 20
6.2.1 Header . . . 20
6.2.2 Each exhibit . . . 20
6.3 Verifying a collection . . . 22
6.3.1 The seal . . . 22
6.3.2 Each exhibit . . . 22
6.4 The investigation log . . . 25
6.5 The artefact files . . . 25
6.5.1 System and accounts . . . 26
6.5.2 Program execution . . . 27
6.5.3 Files and browsing . . . 33
6.5.4 Devices . . . 35
6.5.5 Event logs (`--events`) . . . 35

7 Reading the data **36**
7.1 Timestamps . . . 36
7.2 Account names . . . 36
7.3 GUID names . . . 37
7.4 A missing field is not an empty field . . . 37
7.5 Artefact not collected . . . 37
7.6 Undecoded data . . . 37
7.7 Paths . . . 37

8 Traces left on the examined machine **38**
8.1 What WAC still does . . . 38
8.2 What WAC no longer does . . . 39
8.3 What no tool can avoid . . . 39

9 Known limitations **40**

10 Troubleshooting **41**

A Glossary **42**

CHAPTER 1

Overview

1.1 What WAC is for

WAC collects, from a running Windows computer, the artefacts a digital investigation needs: registry, event logs, Prefetch files, shortcuts and jump lists, scheduled tasks, services, accounts, processes and sessions. It delivers them as **JSON** files, ready to be analysed or fed into a processing pipeline.

It runs from a USB stick, without installation, and writes everything it produces to that stick.

1.2 The guiding principle: a minimal footprint

Examining a machine changes it. Every tool leaves traces: opened files whose last-access date changes, services that write to the logs when solicited, registry keys read through the API. These traces mix with the ones being looked for.

WAC is designed so that what it leaves behind is as close to nothing as the task allows, and so that **whatever it cannot avoid is written down**. In practice:

- the disk is read **directly**, read-only, without the system opening any file: no access date is modified;
- no key of the machine's registry is opened: the hives are copied, then read on the copy;
- no Volume Shadow Copy, no COM or WMI call, no logging service is solicited;
- nothing is written to the examined disk;
- every operation is recorded, with the trace it is expected to leave, in `investigation.json`.

1.3 What WAC does not do

- WAC **does not analyse**: it collects and formats. Interpretation is the analyst's.
- It does not make a full disk image: it extracts the files that hold artefacts.
- It does not capture memory. The only volatile data collected are the process list, the open sessions and the state of the services.

CHAPTER 2

How it works

2.1 Reading the disk directly

WAC opens the volume (`\\.\C:`) read-only, walks the NTFS file table (`$MFT`) itself and extracts the content of the files it needs. The system sees no file being opened: no access date updated, no directory walked, no lock.

This reading handles the NTFS features Windows normally hides:

NTFS compression
Windows 11 compresses, among others, the event log folder. WAC decompresses it itself.

Compact OS (WOF)
The system binaries of Windows 10 and 11 are stored compressed in a hidden stream; seen through the API, they look ordinary. WAC decompresses them (XPRESS variants).

Split attributes
A heavily fragmented file has its information spread over several records; WAC gathers them.

Preallocated files
Beyond the valid data length, NTFS returns zeros, even if the disk still holds remnants of former files. WAC does the same: the extracted content is the one any ordinary acquisition would see, and so is its fingerprint.

2.2 Exhibit store and working copy

A digital exhibit is never examined on itself: a copy is taken and sealed, and the work is done on a second copy. If the analysis damages something, the sealed copy remains and the operation can be redone.

WAC applies this procedure to **every** extracted file, including those it does not modify. The two directories keep their French names in the output:

`exhibits/` — the exhibit store
the raw copies, as read from the volume. This directory is never reopened for writing after extraction. It holds the manifest identifying each exhibit, and the manifest's seal.

`working/` — the working copy
copied from the exhibit store and verified by fingerprint. The only modifications (replay of the hives' transaction logs) are made here, and every collector reads from here.

> **Keep in mind**
> Every extracted file is therefore written **twice** to the collection medium. Plan for the space (see section 3.2).

2.3 Registry hives and transaction logs

A hive copied from a running machine lacks its latest changes: they are still in its transaction logs (`.LOG1`, `.LOG2`). WAC extracts these logs and **replays them onto the working copy**, so that the analysis bears on the machine's actual state.

Everything WAC writes onto a copy can be undone: before the replay, the original content of every replaced page is saved in an undo journal (`<hive>.undo`). The raw copy stays intact in the exhibit store.

2.4 Two-pass extraction

Where each user's hives live is recorded in the `SOFTWARE` hive. WAC therefore first extracts the machine's hives (`SYSTEM`, `SOFTWARE`, `SAM`, `Amcache`), reads the profile list in the copy of `SOFTWARE`, then extracts each profile's hives (`ntuser.dat`, `usrClass.dat`).

Nothing is assumed about `C:`: the system drive is detected, and a profile on another disk (for instance `D:\Users\...`) is read on its own volume.

2.5 What is still read live

A few pieces of information describe the very moment of the collection and are held in no file. They are read live, with a single query each, and recorded as read in the exhibit store (`exhibits\live`), where the conversion reads them back:

- the running **processes** (a kernel snapshot);
- the open **sessions**;
- the **current state of the services** (their configuration is read from the copied hive);
- the **clock** and the boot time;
- the running machine's **settings**: system drive, code page, time zone, volumes and their serial numbers.

CHAPTER 3

Preparing a collection

3.1 Requirements

- Windows 10 or later, 64-bit. WAC checks it at start-up.
- **Administrator rights**: reading a volume directly requires them. WAC checks this too.
- **NTFS**, **FAT** or **exFAT** volumes. A file on another file system (CD-ROM) cannot be read directly; it is reported as unread, never opened another way.

3.2 The collection medium

WAC runs from the collection medium (usually a USB stick) and writes all its results there. Choose an **empty** medium, preferably USB 3: writing is the most expensive part of a collection.

The space needed depends on the mode and the options. Two places fill up: the **exhibit store** (sealed raw copies) and the **working copy**, which takes only what the collectors modify or read during the collection (hives, Prefetch, shortcuts, jump lists, referenced binaries). The binaries acquired by `--collect --binary` and the NTFS metadata are never copied again: the conversion reads them in the exhibit store itself. Measured on the test virtual machine (Windows 11 without added software, 1 volume):

Mode	Exhibit store	Working copy	Time
Full run, `--events --binary` (referenced binaries)	1.2 GB	0.6 GB	3 min 30
Collection only, `--collect --events --binary` (every executable)	2.7 GB	0.3 GB	8 min 30

The heaviest items of the exhibit store:

Content	Exhibit store
NTFS metadata: `$MFT` (380 MB), `$LogFile` (64 MB), `$UsnJrnl:$J` (35 MB, out of 3.6 GB logical: NTFS freed the start, which is not copied)	480 MB
Binaries copied, full run: the 62 referenced binaries not authenticated	250 MB
Binaries copied, collection only: 2,853 not authenticated out of 44,446 read	1.2 GB
Event logs (`--events`)	170 MB
Hives, Prefetch, shortcuts, jump lists	180 to 230 MB

A machine used for a long time, or loaded with software, exceeds these figures: the `$MFT` grows with the number of files, the unauthenticated third-party binaries with the installed software. Allow **at least 5 GB free** for a collection only with `--binary`.

WAC checks the location **before its very first write** and refuses to start:

- if the working directory already holds files — an earlier collection would be analysed instead of this one;
- if the free space is clearly insufficient.

If space runs out while binaries are being collected, they are hashed without being copied, and the investigation log says so.

3.3 Before going on site

- Use a **new output folder** for each machine (option `--output`).
- Note the time of the intervention and the operator's identity: WAC records them too, but an independent record is useful.
- Keep in mind that plugging the stick in and running WAC leave unavoidable traces (chapter 8): they are known and documented, so that they are not mistaken for the activity under study.

CHAPTER 4

Running a collection

4.1 Command line

Open a command prompt **as administrator**, move to the collection medium, then run:

```
WAC.exe [--collect | --convert=folder] [--events] [--binary | --binary-all]
        [--threads=N] [--dump]
        [--output=folder] [--loglevel=N] [--debug]
WAC.exe --update-trust[=folder]
```

Every option is optional and disabled by default.

4.2 Configuration file: `wac.yml`

Version. WAC numbers its versions `<major>.<minor>.<bugfix>`: major for a rework, minor for features added, bugfix for fixes. The version is read in `WAC.exe`'s properties (Details tab), shown at start, and recorded in `investigation.json` (`Tool.Version`): a collection says which version made it. This document describes version 1.4.0.

The last build is in `bin` of the repository: `WAC.exe`, the reference `wac.yml`, the scripts that verify a collection (section 6.3) and `SHA256SUMS` to check them all. Copying `WAC.exe` and `wac.yml` to the collection key is enough; the scripts go with the investigator.

Rather than typing the options at every collection, the procedure is written once in `wac.yml`, next to `WAC.exe`: WAC applies it first, and an option given on the command line overrides it. `--write-config` writes the reference file, commented — the recommended procedure: collection only, unverified binaries collected, event logs included.

```
convert: false          # true: collect and convert here; false: collect only
binary: unverified      # none | unverified | all
threads: 0              # analysis threads for the binaries, 0: automatic
output: output
log_level: 0
dump: false
language: fr            # fr | en: language of the console; the files stay in English
artefacts:              # one switch per artefact; all the hives = registry
  registry: true
  events: true
  scheduled_tasks: true
  services: true
  processes: true
  sessions: true
  network: true
```

```
  prefetch: true
  jump_lists: true
  recent_documents: true
  mft: true             # $MFT: every file, present or deleted, with its dates
  usn_journal: true     # $UsnJrnl:$J: creations, renames, deletions
  log_file: true        # $LogFile and $TxfLog: NTFS and TxF transaction journals, raw
    copy only
  powershell_history: true  # PSReadLine: the commands typed in PowerShell
  activities_cache: true    # Windows Timeline: applications, files, pages, clipboard (
    ActivitiesCache.db)
  setupapi: true        # setupapi.dev.log: device installations
  recycle_bin: true     # $I of the recycle bins: what, by whom, when
  hosts: true           # the hosts file: names diverted to an address
  error_reports: true   # error reports (WER): programs that crashed, with their path
  defender: true        # Defender: logs (MPLog, raw WPP traces), detection history,
    quarantine (copied encrypted)
  startup_folders: true # startup folders: what runs at logon
  zone_identifiers: true  # Zone.Identifier streams: where downloaded files came from
  security: true        # SECURITY hive: LSA secret names, cached logons, audit
  persistence: true     # registry persistence: IFEO, AppInit, Winlogon, LSA, …COM
  office_trust_records: true  # Office trusted documents, macros enabled
  typed_paths: true     # paths typed in Explorer
  word_wheel_query: true  # Explorer searches
  root_certificates: true # root certificates of the machine and users
  internet_settings: true # users' proxy
  firewall: true        # firewall rules and profiles
  wmi: true             # WMI repository read offline: subscriptions (filters, consumers,
    bindings)
  srum: true            # SRUDB.dat: network and resources per application and user, 30 to
     60 days
  bits: true            # qmgr.db: BITS jobs, URLs, destinations, notification command
    lines
  fat_volumes: true     # FAT and exFAT volumes, the suspect's USB keys included: entries,
     deleted ones too
  installed_programs: true  # installed programs (Uninstall) of the machine and of each
    user
  defender_configuration: true  # Microsoft Defender exclusions and protection settings
  explorer_recent_docs: true    # files each user opened (Explorer's RecentDocs)
  mount_points: true            # volumes and shares each user saw (MountPoints2)
  known_networks: true          # networks the machine joined (NetworkList)
  program_usage: true           # programs launched or used (FeatureUsage, JumplistData,
    compatibility, pinned)
  devices: true                 # USB and SCSI devices with their dates, portable devices,
     volume labels
  network_shares: true          # the machine's network shares, path and rights
  network_configuration: true   # network adapters, last IPv4/IPv6 settings, DHCP leases (
    offline)
  event_log_configuration: true # state of each event log and boot ETW session: enabled,
    size, retention, access (offline)
  shortcuts: true               # shortcuts outside Recent: Start menu, desktops, Quick
    Launch, task bar
  security_descriptors: true    # $Secure:$SDS: owner and rights of every file, each
    descriptor once
```

```
  ntfs_volumes: true            # $Boot and $UsnJrnl:$Max: serial and geometry, creation
    of the journal
  dpapi_master_keys: true       # DPAPI master keys copied encrypted; JSON: owner, GUID,
    dates, never the key
  user_shell_folders: true      # where each user's desktop, documents, downloads really
    are
  web_client: true              # Chromium browsers and Firefox: visits, downloads,
    searches; their databases copied raw
```

- **Strict**: an unknown key, a value outside the ones allowed (`true`/`false` only, not `yes`), a duplicate key, a list: the collection is refused before it starts, with the line at fault. A key left out keeps its default.
- **Recorded**: the file's path, SHA-256 and content go into `investigation.json` (`Tool.Configuration`) — the procedure applied is part of the evidence.
- An artefact switched off is not read at all, and its JSON file says so (`CollectionStatus NotRequested`, with the key): never mistaken for a failure or an absence of traces.
- `--config=file` reads another file; `--full` collects and converts here although the file says `convert: false`.
- Without `wac.yml`, WAC behaves as before: full run, options as given.

4.3 Options

Option	Effect
`--collect`	Collection only, on the examined machine: live observations and raw extraction into the sealed exhibit store; nothing is converted. See chapter 5.
`--convert=folder`	Conversion only, on an analysis workstation, of the collection made with `--collect` in that folder, after checking the seal and every fingerprint. See chapter 5.
`--events`	Extracts and parses the event logs (`.evtx`), and rebuilds each event's readable message from its provider's resources. Adds about 250 MB of copies.
`--binary`	Fingerprints (MD5, SHA-1, SHA-256) every file referenced by an artefact, by reading the disk directly. Referenced executables, libraries, drivers and scripts are also **collected** into the exhibit store. See section 4.5.
`--binary-all`	Like `--binary`, but **every** executable is collected, authenticated or not; the signature is checked all the same, and the manifest says for each whether it holds. For the cases where the content itself must be in the exhibit store.

Option	Effect
`--threads=N`	Number of threads analysing the executables of `--collect --binary` (PE, Authenticode, catalogs, fingerprints). By default, the processor's threads minus one, the disk reading keeping its own. The result does not depend on it: the manifest is the same.
`--update-trust [=folder]`	On the **analysis workstation**, **before** the collection, and alone: downloads from Microsoft the list of trusted roots (`authroot.stl`), that of disallowed certificates (`disallowedcert.stl`) and every root, checks their signatures and fingerprints, together with the vulnerable driver lists of Microsoft and LOLDrivers (not signed: authenticated by HTTPS alone) and the revocation lists (CRL) of the code signing authorities the CCADB lists, of Microsoft's own, and of those an earlier collection found missing (`wanted-crls.txt`), and writes them into `folder` (by default `trust`, next to `WAC.exe`), to be carried on the key with `WAC.exe`. The only mode that uses the network: the network library is loaded there alone. Refused with any other option.
`--dump`	Adds the raw hexadecimal value of shellbag and `.lnk` shortcut items, for later decoding.
`--output=folder`	Output folder, relative to the current directory. Default: `output`.
`--language=fr|en`	Language of the console: steps, help, error messages. French by default; `language:` in `wac.yml` sets it too. The JSON files, `investigation.json` and the text log stay in English whatever the language: they are evidence read by tools, and a collection gives the same files whoever the operator.
`--loglevel=N`	Enables a detailed text log in the current directory, named after the program run (`WAC.exe.log` for `WAC.exe`): 0 none, 1 per artefact type, 2 per artefact, 3 per sub-function (debugging).
`--debug`	Prints the NTFS reader's trace on standard error (path resolution, index blocks, cluster runs). For diagnosis only.
`--help, /?`	Shows the help.

4.4 Examples

Standard collection:

```
E:\> WAC.exe --output=accounting-pc-01
```

Full collection, with logs, binaries and a detailed log:

```
E:\> WAC.exe --events --binary --output=accounting-pc-01 --loglevel=2
```

4.5 The `--binary` option

A fingerprint is enough to query a public database (VirusTotal, for instance) **without sending it the file**. But it says nothing about a binary nobody knows — precisely the one that matters to the investigation — and a binary left behind may be gone by the time a detection comes in. This is why `--binary` also copies potential payloads:

- **collected**: `exe`, `dll`, `sys`, `ocx`, `cpl`, `scr`, `drv`, `efi`, `com`, `msi` and the scripts `ps1`, `psm1`, `bat`, `cmd`, `vbs`, `vbe`, `js`, `jse`, `wsf`, `wsh`, `hta`;
- **collected as well**: Office documents **able to carry macros**, an intrusion vector as common as an executable — legacy binary formats (`doc`, `xls`, `ppt`, `pps`, `dot`, `xlt`, `pot`), "m" formats (`docm`, `dotm`, `xlsm`, `xltm`, `pptm`, `potm`, `ppsm`), `xlsb`, add-ins (`xla`, `xlam`, `xll`, `wll`, `ppa`, `ppam`), Publisher (`pub`), Visio (`vsd`, `vsdm`, `vstm`, `vssm`) and Access (`mdb`, `accdb`, `accde`). They appear in the traces as targets of shortcuts or jump lists, or among the files loaded by Word or Excel in their Prefetch;
- **only hashed**: other documents and data files, including `docx`, `xlsx` and `pptx`, which cannot contain macros. Copying them would turn the collection into a copy of the user's files.

The referenced files come from processes, services (binary and DLL), scheduled tasks, files loaded by programs (Prefetch), Shimcache, Amcache, and targets of shortcuts and jump lists. Each file is read only once, however many artefacts cite it, and **identical content is stored only once**: the other paths appear in the manifest as exhibits sharing that content (`SharedExhibit`).

Authentic Microsoft binaries: hashed, not collected

Most referenced binaries are Windows components, identical on every machine of the same build. Their origin can be proven on the machine itself, with no list to carry: Windows' **catalogs**, signed by Microsoft, list the fingerprint of every system file. WAC reads those of `System32\CatRoot`, those of the components (`WinSxS\Catalogs`) and those of the servicing packages (`servicing\Packages`), over 13,000 on Windows 11. Individually signed binaries (Edge, OneDrive, Office, .NET) carry an **embedded signature**. A binary authenticated this way is hashed without being collected; every other one is collected — including a system binary replaced by an attacker, whose fingerprint no longer matches.

This check leaves **no trace**: catalogs and binaries are read directly from the disk, and the cryptographic verification is done in memory by WAC, up to Microsoft root certificates embedded in the executable. No service is solicited, the machine's certificate store is not consulted, and revocation is not checked online.

Only signers Microsoft uses for its own code are accepted ("Microsoft Windows", "Microsoft Corporation", "Microsoft Windows Publisher" and ".NET"). **Third-party** drivers certified by Microsoft (signers "Hardware Compatibility Publisher" and "Early Launch Anti-malware Publisher") are still collected: a vulnerable but signed driver is a classic attack path.

Store applications: verified through their package

The files of a Store application are not signed one by one: their **package** is. Its folder holds the signature (`AppxSignature.p7x`) and the block map (`AppxBlockMap.xml`), which gives the SHA-256 of every 64 KiB block of every file. WAC verifies the signature, which must reach a Microsoft root (the Store signs the packages it distributes), checks that it does cover that map, then compares every block read with the map. A matching file is hashed without being collected; a modified file, or one added to the folder after installation, is collected, and so is every file of a package signed outside the Microsoft chain.

The catalogs, package signatures and block maps that justified not collecting a file go into the exhibit store, so that a third party can redo the check. In the artefact files, the `Signature` field (or `ServiceDllSignature`, `SignatureTarget`…) states the authentication used, for instance "`Microsoft (catalog C:\Windows\….cat)`", "`Microsoft (embedded signature)`" or "`Package (package C:\Program Files\WindowsApps\…)`".

Scripts are authenticated the same way: by Windows' catalogs and, for PowerShell (`ps1`, `psm1`, `psd1`, `ps1xml`…), by their embedded signature block (`# SIG # Begin signature block`). A Windows Script Host script (`vbs`, `js`, `wsf`) signed only within its text, and not by a catalog, is still collected.

On the test machine, this check brings collection down from 2,131 binaries (3.3 GB) to 92 (335 MB), without lengthening the collection; the scripts still collected are the unsigned ones.

Since reading is direct, collecting leaves no more trace than hashing — it only takes space on the medium.

4.6 What the screen shows

WAC displays each step, followed by `OK` or an error message. A failed step does not stop the collection: whatever is accessible is gathered, and whatever is missing is recorded.

```
[PREREQUISITE VERIFICATION]
 - Check OS >= Windows 10 : OK
 - Check administrator rights : OK
 - Checking the collection medium : OK
[SEARCHING FOR ARTIFACTS IN WINDOWS API]
 - Extraction of SESSIONS : OK
 - Extraction of PROCESS : OK
 - Extraction of NETWORK STATE : OK
[RAW EXTRACTION]
 - Extracting system hives (raw NTFS) : OK
 - loading the HKLM\SYSTEM key : OK
 - loading the HKLM\SOFTWARE key : OK
 - Listing USER PROFILES (SOFTWARE hive) : OK
 - Extracting user hives (raw NTFS) : OK
 - Extracting file artefacts (raw NTFS) : OK
[SEARCHING FOR ARTIFACTS IN THE REGISTRY]
 - Extracting USERASSIST Registry Keys : OK
   ...
[SEARCHING FOR ARTIFACTS IN FILES]
```

```
  - Extracting PREFETCHS : OK
    ...
[SEARCHING FOR ARTIFACTS IN EVENT LOGS]
 - Extraction of EVENTS : OK
[EXHIBIT STORE]
 - Copying collected binaries to the working directory : OK
 - Sealing the exhibit store (manifest + SHA-256) : OK
[INVESTIGATION LOG]
 - Writing investigation.json : OK
END, Time elapsed : 309 s
```

This is the display with `--language=en`, or `language: en` in `wac.yml`: by default the console is in French.

Sealing the exhibit store is the **last** operation on it: every exhibit, including those added while the event logs are processed, is covered by the manifest.

4.7 Duration

The duration depends mostly on the hardware (USB 2 or 3, disk, processor) and the options. On the test virtual machine (NVMe disk): about 70 s with `--events`, about 310 s with `--events` and `--binary`. On a USB stick, writing dominates.

CHAPTER 5

Collect here, convert elsewhere

5.1 Why separate

A run of WAC does two different things:

1. **the collection**: observing what only a running system shows (processes, sessions, service states, clock), reading the disk directly, copying the evidence into a sealed exhibit store. It **needs** the examined machine;
2. **the conversion**: parsing thousands of copied files (hives, event logs, Prefetch, shortcuts…) into JSON. It only needs the exhibit store, **not** the machine.

Without a mode option, both run on the examined machine. The conversion adds work there that nothing requires: minutes of processing in WAC's memory, which Windows may page out to `pagefile.sys`, on the very disk under examination; a process present for longer, seen and logged by the machine's security software; if it crashes, a Windows error report written on the machine. It reads neither the machine's files nor its registry, only the copies on the collection medium, but it runs there.

That is exactly what good practice asks to avoid:

- **ISO/IEC 27037** covers the identification, collection and preservation of digital evidence, **ISO/IEC 27042** its analysis: two distinct stages;
- **ACPO principle 1**: no action should change the data of the examined machine; every operation run there must be necessary;
- **ACPO principle 3**: an independent third party must be able to repeat the process and reach the same result.

Separating the two stages brings three things:

- **fewer traces** on the examined machine: only the collection runs there;
- **evidence checked before any analysis**: `--convert` first verifies the seal and the SHA-256 of every exhibit, and refuses a retouched collection;
- **a reproducible analysis**: two conversions of one collection give byte-identical JSON files, and the analysis can be redone later, with a corrected version of WAC or by a third party, without going back to the machine.

5.2 Which mode

- **By default: `--collect` on the machine, `--convert` on the analysis workstation.** It is the procedure that leaves the fewest traces on the examined machine, keeps the evidence sealed, and allows the analysis to be redone and checked.
- **Full run** (no mode option): for specific cases, to be stated in the report — a triage whose results are needed on the spot with no analysis workstation, a machine to be released within minutes. Its conversion reads only the copies on the collection medium, never the machine's files or registry, but it runs on it: minutes of processing in memory (which Windows may page to `pagefile.sys`), a longer process, a Windows error report if it crashes.
- **Scope of `--binary`**: in a full run, only the files **cited by the artefacts** are read; a binary on the disk that no artefact cites is not examined. With `--collect`, it is **every executable of every fixed NTFS volume**, cited or not. The collection alone takes longer for that reason (8 min 30 against 3 min 30 on the test machine): that time is reading, which writes nothing to the machine.
- **`--binary-all`**: when the content of every executable must be in the exhibit store (malware analysis, request for the files themselves), for about 20 GB.

5.3 The two commands

On the examined machine, as administrator:

```
E:\> WAC.exe --collect --events --binary --output=accounting-pc-01
```

On the analysis workstation, with no particular rights, with the same `--events` and `--binary` options as the collection:

```
D:\> WAC.exe --convert=E:\accounting-pc-01 --events --binary
```

The JSON files are written into the collection's folder, next to the exhibit store. `--convert` excludes `--collect` and `--output`.

5.4 What `--collect` takes

Since the conversion can no longer ask the machine anything, the collection takes **everything it will read**:

- the **live observations**, recorded as read in `exhibits\live` and sealed like any exhibit: processes, sessions, service states, clock and boot, and the machine's settings (system drive, code page, time zone, volumes and their serial numbers);
- the hives and the file artefacts, as usual;

- with `--events`, the resource files of **every** declared event provider (binary, parameter file, `.mui` satellites): which of them are needed is only known at conversion. About 650 MB on Windows 11;
- with `--binary`, **every** executable, library, driver, script and macro document of **every fixed NTFS volume** — a profile on `D:` included —, read directly and verified in memory, as in a full run (section 4.5):
  - authenticated (catalog, embedded signature, Store package), or a third-party binary whose signature holds against the trust set of the key (`--update-trust`: chain to a root Microsoft trusts for code signing, valid at the signing time, not revoked, not a vulnerable driver): **not copied**; the manifest keeps its path, its Authenticode digests and the verdict (`ContentStored false`); without a usable trust set, every third-party binary is copied, as the investigation log says;
  - not authenticated: **copied**, with its three fingerprints MD5, SHA-1, SHA-256, its Authenticode digest and the reason;
  - Windows differential file (`PA30`/`PA31`, the `r` and `f` folders of `WinSxS`): not an executable; hashed, not copied.

  A content is stored once, and the other names of a hard link (`System32` and `WinSxS`) are recorded in the manifest like the first. Removable volumes, the collection medium among them, and the memory files (`pagefile.sys`, `swapfile.sys`, `hiberfil.sys`) are left out.

> **Warning**
> With `--binary`, the collection reads every volume: on the test virtual machine (Windows 11 without added software), 44,446 executables read, of which 41,593 authenticated (or third-party with a valid signature) not copied and 2,853 copied (1.2 GB); the whole collection takes 8 minutes 30 and the exhibit store, event logs and NTFS metadata included, weighs 2.7 GB (see section 3.2). If space runs short, the remaining files are hashed without being copied, and the manifest says so.

No artefact JSON file is written: the folder holds only the sealed exhibit store, the working directory and `investigation.json`.

5.5 What `--convert` does

1. **Check**, before any read: the seal must carry the manifest's real fingerprint, and every exhibit its SHA-256 fingerprint. A single altered exhibit, and the conversion is **refused**; `conversion.json` gives the reason.
2. **Rebuild of the working directory** from the exhibit store: the former one is deleted rather than trusted, the copies are verified against the manifest and the hives repaired, as at collection time. The executables and the providers' resources are read in the exhibit store itself, without a copy.
3. **Conversion**, from the exhibits alone: nothing is read from the converting machine — neither its disk, nor its registry, nor its time zone.

The exhibit store is never modified and its seal remains the collection's. `investigation.json` remains the collection's log; `conversion.json` is the conversion's (operator, workstation, operations). Two conversions of one collection give identical JSON files, which the test harness checks on every cycle.

5.6 Limitations

- **Macro documents** (`doc`, `docm`, `xls`, `xlsm`, `ppt`…, list in section 4.5) are handled like executables: read on every fixed volume and **collected**, unless they are, byte for byte, a file shipped by Microsoft and listed in a catalog — which a document written or modified by anyone cannot be.
- A cited file that can carry neither code nor macro (`docx`, `xlsx`, `pdf`, data) is not collected: at conversion it therefore has **no fingerprint**, whereas the full run computes it on the disk.
- An authenticated executable is not copied: at conversion it has its Authenticode digest and its verdict, but no MD5, SHA-1 or SHA-256 — those are computed only for what is collected.
- An executable on a removable volume is not collected by `--collect --binary`.
- The `--events` and `--binary` options of the conversion must be those of the collection: what the collection did not take cannot be converted.

CHAPTER 6

The results

6.1 Layout

```
<output>/
  exhibits/            exhibit store: raw copies, never rewritten
    MANIFEST.json     identifies each exhibit and the collection
    MANIFEST.sha256   the manifest's fingerprint: its seal
    Windows/...        the exhibits, under their original paths
    Users/...
  working/             working copies (replayed hives)
  investigation.json   the collection log
  *.json               the artefacts
```

A file from a volume other than the system one is stored under `_volume_D\...`; the JSON always gives the original path, with its real drive letter.

6.2 The exhibit store manifest

`exhibits/MANIFEST.json` identifies the collection and each exhibit.

6.2.1 Header

`Tool`
: tool name, command line, build date.

`Host`
: machine name, system drive, time zone of the suspect (read from its `SYSTEM` hive) and of the collecting machine.

`Operator`
: account that ran the collection, and whether it was elevated.

`Custody`
: procedure statement, start of extraction, time the manifest was written, number of exhibits and failures, total size, hash algorithms, volumes read (letter, serial number, file system).

6.2.2 Each exhibit

Field	Meaning
`SourcePath`	Original path, with drive letter.
`ExhibitPath`	Location of the raw copy, relative to the output folder.
`Method`	Collection method (direct NTFS reading, volume, purpose).
`Result`	`OK`, or the error code and message. Failures are listed in the manifest: a missing exhibit would read as one never looked for.
`MD5, SHA1, SHA256`	The three fingerprints, computed while writing: they bear on what was read from the volume.
`Bytes`	Extracted size.
`DeclaredBytes`	Size declared by NTFS, present only when it differs from the extracted size (truncated extraction).
`ValidDataBytes`	Valid data length of a preallocated file (event logs): beyond it, the exhibit holds zeros by definition.
`SharedExhibit`	`true` when the content, identical byte for byte, is already stored under another exhibit: `ExhibitPath` points to it. The source remains an exhibit in its own right, with its own path, `$MFT` entry and timestamps.
`MftEntry`	NTFS record number: identifies the file independently of its name.
`Extracted(Utc)`	When this exhibit was extracted.
`SourceCreated, SourceModified, SourceMftModified, SourceAccessed`	The four NTFS timestamps of the original file (`$STANDARD_INFORMATION`), in UTC and in the suspect's local time. They, not the dates of the copies, date the file.
`SourceFileName Created, Modified, MftModified, Accessed`	The four timestamps of the `$FILE_NAME` attribute of the name read. NTFS keeps them apart, and a change of the dates by a program leaves them untouched: a `SourceCreated` earlier than `SourceFileNameCreated` is the classic sign of dates forged after the fact (*timestomping*).
`ContentStored`	`false`: authenticated binary read but not copied (`--collect --binary`); it has no `ExhibitPath`.
`AuthenticodeSHA1, AuthenticodeSHA256`	Authenticode digests of a PE, those its signature and the catalogs cover; the only digests of an authenticated binary that was not copied.
`SignatureVerified`	Outcome of an executable's signature check: `true` with `Signature` (what authenticates it), `false` with `SignatureReason` (why not).

Field	Meaning
`PeTimeDateStamp`, `PeSizeOfImage`, `SymbolServerKey`	Build of a PE and the key (`name/TIMESTAMPsize/name`) under which Microsoft's symbol server keeps it: an authentic Microsoft binary that was not copied can be fetched again from it, identical, and its fingerprint checked.

> **Why three fingerprints**
> MD5 collisions have been producible at will since 2008, and SHA-1 ones since 2017: a single fingerprint no longer settles an identity dispute. Three independent ones do.

6.3 Verifying a collection

6.3.1 The seal

The manifest cannot hold its own fingerprint: it is in `MANIFEST.sha256`. On Linux:

```
$ cd output/exhibits
$ sha256sum -c MANIFEST.sha256
MANIFEST.json: OK
```

6.3.2 Each exhibit

Two scripts, delivered with `WAC.exe` in `bin`, check the seal, then the SHA-256 of every exhibit: `verify-exhibits.ps1` on Windows, `verify-exhibits.sh` on Linux. Their French counterparts, `verifier-consigne.ps1` and `verifier-consigne.sh`, make the same checks with French messages.

Verify from a Linux machine. This is the recommended way: the collection may hold what infected the examined machine, and a Linux investigation machine is a technological break — a Windows executable or macro document of the collection cannot run there. The script needs only `bash`, the coreutils and `python3` (which reads the JSON); the fingerprints are computed by `sha256sum`, not by WAC:

```
$ bash verify-exhibits.sh /media/usb/output
Seal verified (CEB489A143AE29EB9E3BC836F3D565B744F036AE1F3A72A3447408F31F8A5B67)
1617 exhibit(s) verified, 0 mismatch(es)
0 binarie(s) authenticated and fingerprinted only (no copy to verify)
```

The manifest comes from the examined machine: both scripts refuse an exhibit path that leaves the collection (`..`, absolute path, and for the `.sh` a symbolic link), and count it as a mismatch without reading it.

From Windows:

```
PS> powershell -ExecutionPolicy Bypass -File verify-exhibits.ps1 -Output E:\output
Seal verified (CEB489A143AE29EB9E3BC836F3D565B744F036AE1F3A72A3447408F31F8A5B67)
1617 exhibit(s) verified, 0 mismatch(es)
```

Their exit code is 0 if everything matches, 1 otherwise.

```
#!/usr/bin/env bash
# verify-exhibits.sh — checks the seal and every exhibit of a WAC collection,
# from a Linux investigation machine: the collection is examined away from the
# Windows machine it comes from, which it cannot infect there.
# Usage: bash verify-exhibits.sh /media/usb/output
# Exit code: 0 if everything matches, 1 otherwise.
# Needs: bash, coreutils (sha256sum, realpath), python3 (reads the JSON only).
#
# The fingerprints are computed by sha256sum, not by WAC: a verification that
# does not rely on the tool that produced the collection.
#
# The manifest comes from the examined machine, so it is treated as hostile:
# an exhibit path that leaves the collection (.., absolute path, symbolic
# link) is refused and counted as a mismatch, never read.

set -u
output="${1:-./output}"
manifest="$output/exhibits/MANIFEST.json"
seal="$output/exhibits/MANIFEST.sha256"

for tool in sha256sum realpath python3; do
    command -v "$tool" >/dev/null || { echo "MISSING TOOL: $tool"; exit 1; }
done
[ -f "$manifest" ] && [ -f "$seal" ] || { echo "NO MANIFEST: $manifest or $seal absent";
    exit 1; }
root="$(realpath -e -- "$output")" || exit 1

# 1. The seal: the manifest's fingerprint must be the one recorded beside it.
recorded="$(head -c 64 -- "$seal" | tr 'a-f' 'A-F')"
actual="$(sha256sum -- "$manifest" | head -c 64 | tr 'a-f' 'A-F')"
if [ "$recorded" != "$actual" ]; then echo "SEAL MISMATCH: the manifest was modified";
    exit 1; fi
echo "Seal verified ($actual)"

# 2. Every exhibit: its SHA-256 must be the one in the manifest.
# python3 lists the exhibits kept with their content, as NUL-separated
# "S, SHA-256, path" records, and one "F" record per authenticated binary that
# was not copied (--collect --binary): no file to verify. NUL cannot occur in
# a path, whatever the manifest holds.
list_exhibits() {
    python3 - "$manifest" <<'PYTHON'
import json, sys
with open(sys.argv[1], encoding="utf-8-sig") as f:
    items = json.load(f).get("Items") or []
out = sys.stdout.buffer
for item in items:
    if not isinstance(item, dict) or item.get("Result") != "OK":
        continue
```

```
    if item.get("ContentStored") is False:
        out.write(b"F\0")
        continue
    sha = str(item.get("SHA256") or "").replace("\0", "")
    path = str(item.get("ExhibitPath") or "").replace("\0", "")
    out.write(b"S\0" + sha.encode() + b"\0" + path.encode("utf-8", "surrogateescape") + b
    "\0")
PYTHON
}

ok=0; ko=0; fingerprint_only=0
while IFS= read -r -d '' kind; do
    if [ "$kind" = F ]; then fingerprint_only=$((fingerprint_only + 1)); continue; fi
    IFS= read -r -d '' expected || break
    IFS= read -r -d '' exhibit || break
    # The manifest writes Windows paths, relative to the output folder.
    relative="${exhibit//\\//}"
    file="$(realpath -e -- "$root/$relative" 2>/dev/null)" || file=""
    case "$relative" in
        ""|/*) file="" ;;
    esac
    case "$file" in
        "$root"/*) ;;
        *) ko=$((ko + 1)); echo "MISMATCH: $exhibit"; continue ;;
    esac
    hash="$(sha256sum -- "$file" | head -c 64 | tr 'a-f' 'A-F')"
    if [ -n "$expected" ] && [ "$hash" = "$(printf '%s' "$expected" | tr 'a-f' 'A-F')" ];
    then
        ok=$((ok + 1))
    else
        ko=$((ko + 1)); echo "MISMATCH: $exhibit"
    fi
done < <(list_exhibits || printf 'S\0\0unreadable manifest\0')

echo "$ok exhibit(s) verified, $ko mismatch(es)"
echo "$fingerprint_only binarie(s) authenticated and fingerprinted only (no copy to verify
    )"
[ "$ko" -eq 0 ] || exit 1
```

```
# verify-exhibits.ps1 — checks the seal and every exhibit of a WAC collection.
# Usage: powershell -ExecutionPolicy Bypass -File verify-exhibits.ps1 -Output E:\output
# Exit code: 0 if everything matches, 1 otherwise.

param([string]$Output = '.\output')

# 1. The seal: the manifest's fingerprint must be the one recorded beside it.
$recorded = ((Get-Content -Raw "$Output\exhibits\MANIFEST.sha256") -split '\s+')[0]
$actual   = (Get-FileHash -Algorithm SHA256 "$Output\exhibits\MANIFEST.json").Hash
if ($recorded -ne $actual) { Write-Output "SEAL MISMATCH: the manifest was modified"; exit
     1 }
Write-Output "Seal verified ($actual)"

# 2. Every exhibit: its SHA-256 must be the one in the manifest.
```

```
$m = Get-Content -Raw -Encoding UTF8 "$Output\exhibits\MANIFEST.json" | ConvertFrom-Json
# The manifest comes from the examined machine: an exhibit path that leaves
# the collection (.., absolute path) is refused, never read.
$root = (Resolve-Path -LiteralPath $Output).ProviderPath.TrimEnd('\') + '\'
$ok = 0; $ko = 0; $fingerprintOnly = 0
foreach ($p in ($m.Items | Where-Object { $_.Result -eq 'OK' })) {
    # An authenticated binary that was not copied (--collect --binary): no file.
    if ($p.ContentStored -eq $false) { $fingerprintOnly++; continue }
    try { $f = [IO.Path]::GetFullPath((Join-Path $root $p.ExhibitPath)) } catch { $f = ''
    }
    $h = $null
    if ($f.StartsWith($root, [StringComparison]::OrdinalIgnoreCase)) {
        $h = (Get-FileHash -Algorithm SHA256 -LiteralPath $f -ErrorAction SilentlyContinue
    ).Hash
    }
    if ($h -and $h -eq $p.SHA256) { $ok++ } else { $ko++; Write-Output "MISMATCH: $($p.
    ExhibitPath)" }
}
Write-Output "$ok exhibit(s) verified, $ko mismatch(es)"
Write-Output "$fingerprintOnly binarie(s) authenticated and fingerprinted only (no copy to
     verify)"
if ($ko) { exit 1 }
```

6.4 The investigation log

`investigation.json` traces the collection: tool and command line, machine and time zones, operator, start, end and duration, then the **ordered** list of operations. For each one:

`Seq`, `TimestampUtc`, `TimestampLocal`
: rank and time;

`Operation`, `Target`
: what was done, and on what;

`Result`
: the outcome;

`Footprint`
: the trace this operation is expected to leave on the examined machine.

This log makes it possible to compare, collection by collection, the expected traces with those actually observed. It also flags a difference between the suspect's time zone and the collecting machine's.

6.5 The artefact files

One file per artefact. The fields described are those emitted; a field without a value is not emitted (see chapter 7).

6.5.1 System and accounts

`OperatingSystem.json`
machine name and domain, Windows edition and version, installation date, current time, **boot time**, uptime, time zone. `BootTimeSource` says where the boot time comes from (kernel, or fallback estimate); `ClockAdjustedSinceBootMs` gives the total clock corrections since boot — a few seconds is routine resynchronisation, a large value means a clock changed. `ShutdownTime`: the last orderly shutdown. `NtfsLastAccessUpdated`: false, NTFS does not update last access dates, which then say nothing. `Settings`: the settings that decide how the other artefacts read, as stored — NTFS last access and short names, Prefetch on, page file and memory dumps, control sets, remote desktop, Windows timeline, last user who logged on.

`security.json`
`SECURITY` hive: **names** of the LSA secrets and the dates of their values (`DefaultPassword` reveals an automatic logon's stored password, `_SC_<service>` a service running under an account), number of cached domain logon slots and how many are used, audit policy (`PolAdtEv`, as bytes: its layout is not published), and the account rights (`AccountRights`, read from `Policy\Accounts`): for each account or group, its privileges (`Privileges`, e.g. `SeDebugPrivilege`, `SeBackupPrivilege`) and its logon rights (`LogonRights`). A sensitive privilege granted to an unexpected account is a persistence. **No secret's value** is read, nor any cached credential. The `config\RegBack` copies of the hives are collected as they are.

`users.json`
local accounts, read from the `SAM` hive: SID and RID, state, flags, logon and failure counts, dates of last logon, last failure, password change and account change, profile, and the password's NT and LM hashes (`NtHash`, `LmHash`, hex), with their history (`NtHashHistory`, `LmHashHistory`). A hash is one-way: it names a password without revealing it. A weak or empty password used by an intruder reads as a legitimate logon; **WAC judges nothing**, it converts, and WAE confronts the hashes, offline, with those of known weak passwords. The hashes are decrypted on the spot (the boot key from `SYSTEM`, the hashed boot key from the `SAM`, the AES scheme of Windows 10 and later or the older RC4, then a DES layer keyed on the RID). If the account carries password-reset security questions (`SecurityQuestions`, the `ResetData` value), they are converted as they are, questions AND answers: stored in the clear, a trivial answer is a weakness WAE judges like a weak password.

`local_groups.json`
local groups read from the `SAM`: the built-in groups (`Scope Builtin`, `S-1-5-32-…`) and the groups created on the machine (`Local`), each with its RID, SID, name, description and direct members (SID and name resolved offline). Membership is what says who is an administrator or a remote desktop user; an account an intruder adds to a privileged group shows up here.

`Sessions.json`
sessions open at collection time: user, logon type, authentication package, start time.

`processes.json`
running processes: name, identifiers (and parent), owner, session, loaded modules and, with `--binary`, the executable's fingerprints.

`network.json`
: network state at collection time: `Connections` (TCP and UDP, IPv4 and IPv6: addresses, ports, state, owning process and its name, creation time local and UTC), `Neighbours` (ARP and NDP caches: the machines of the local network spoken with lately), `Interfaces` (addresses, gateways, DNS servers, DHCP) and `Routes`. `TablesNotRead` names a table Windows refused to give, which is not an empty table.

`services.json`
: services and drivers, configuration read from the hive: type, start mode, account, binary and hosted DLL, failure command, dependencies, last write of the key; current state; with `--binary`, fingerprints of the binary and of the DLL.

`ScheduledTasks.json`
: scheduled tasks, read from their XML definitions: author, run-as account, state, last run and its result, triggers and actions (with `--binary`, fingerprints of the command).

6.5.2 Program execution

`prefetchs.json`
: Prefetch files: program, run count and times, volumes, and list of loaded files with their `$MFT` reference (and their fingerprints with `--binary`).

`amcache_application_files.json`, `amcache_applications.json`, `amcache_inventory.json`
: Amcache inventory, table by table and value by value, with the last write time of each entry: the executables (path, publisher, version, size, link date, `Sha1`), the installed applications, and in the third file the drivers (with their `Sha1`), devices, containers, driver packages and shortcuts, each with its `Table`. **Beware**: for a file larger than 30 MiB, Amcache hashes only the first 31,457,280 bytes; `Sha1CoversBytes` says so, and that SHA-1 does not compare with the whole file's.

`shimcache.json`
: compatibility cache: path and modification date of the file, position in the cache (`CachePosition`, 0 for the most recent). An entry proves the binary was present, not that it ran. The execution flag other tools publish is *not* collected: measured against Prefetch, its "yes" is nearly always right, but its "no" is wrong for 9 executed binaries out of 10 — it would lead to conclude that a program never ran.

`bams.json`
: last execution per program and per user (Background Activity Moderator). The hives being read from the disk, an execution of the last minute or so may be missing: Windows keeps a registry change in memory before writing it (56 to 82 s measured on the test machine, which `RegFlushKey` did not bring forward), and WAC does not force that write, which would modify the examined disk.

`userassists.json`
: programs launched from the user interface: count, number and duration in the foreground (`FocusTimeMs`), last execution.

`muicache.json`
: display names of executed programs.

`run.json`
programs started at boot or logon (Run and RunOnce keys), with the key's last write: in SOFTWARE (the whole machine), in the DEFAULT hive (the system account) and in each profile's NTUSER.DAT (`Hive`). `Type` `REG_EXPAND_SZ`: the command holds variables (`%SystemRoot%`…), given as written. Also: the 32-bit programs' Run keys (`SOFTWARE (32-bit)`); `StartupApproved` (`Key` `StartupApproved\Run`, `Run32`, `StartupFolder`), an entry's state as the user set it in Task Manager or Settings: `Status`, the first 4 bytes as stored (their meaning is not measured, so not interpreted), and `Changed`, the instant of the change; `RunNotification`, a startup entry Windows noticed (`Notification`). A name without a matching Run value is an entry since removed.

`persistence.json`
registry persistence beyond the Run keys: IFEO (`Debugger`, `GlobalFlag`), SilentProcessExit, AppInit_DLLs and AppCertDlls, Winlogon (`Shell`, `Userinit`, `Taskman`, `Notify`), LSA packages, print monitors, `BootExecute`, Active Setup, NetSh helpers, the open commands of the executable file types (`exefile`, `batfile`, `cmdfile`…, machine and user), and COM hijacking in each user's classes (UsrClass.dat). For each value: mechanism (`Mechanism`), hive, key, name, type, value as written, the key's last write, and with `--binary` the fingerprints of the file it names. Every value is given, Windows' own included: telling the added from the default is the analysis's work. BITS jobs are not read yet.

`office_trust_records.json`
Office documents a user trusted ("Enable content"): application, version, document, date; `MacrosEnabled` when macros were enabled — the door of most macro infections.

`typed_paths.json`, `word_wheel_query.json`
paths typed in Explorer's address bar and terms searched there, per user and position (`MruPosition`, 0 = the most recent, as in `shellbags.json`).

`root_certificates.json`
certificates of the machine's and each user's root stores: subject, issuer, validity; a root added makes Windows trust whatever it signs (TLS interception, signed malware). `ThumbprintMismatch` flags a certificate whose SHA-1 is not its key's name.

`internet_settings.json`
each user's proxy and automatic configuration script.

`firewall.json`
firewall rules (name, action, direction, ports, program; every field in `Fields`) and the domain, private and public profile settings.

`installed_programs.json`
installed programs, read in the `Uninstall` keys of the machine (64- and 32-bit) and of each user and Windows Installer products (`Scope` `MSI product`, with the cached package `LocalPackage`): all their values (name, publisher, version, folder, uninstall command, `InstallDate` — the day only, as written) and the key's write time. A remote access tool installed, then uninstalled, often leaves its trace there.

`defender_configuration.json`
Microsoft Defender's configuration read in the registry: the **exclusions** (paths, extensions, processes, addresses; `Source` `Settings` or `Policy`), which an intruder adds so

that their tools are not scanned, and the values that set or switch off protection (real time, tamper protection, sample submission…).

`explorer_recent_docs.json`
files each user opened, as Explorer keeps them (`RecentDocs`): by extension (`Extension`) and for every extension together, with their rank (`MruPosition`, 0 for the most recent), the shell item of the shortcut made in Recent, and the list's write time, which is when the file at position 0 was opened. The name outlives the file and the shortcut.

`mount_points.json`
volumes and network shares each user's Explorer saw (`MountPoints2`): volume (GUID, to match with `mounted_device.json` for a USB key) or share (`Share`, `\\server\share`), the entry's write time — the last time Explorer saw it —, its values, and its subkeys with their command (`Shell\AutoRun\command`: an old persistence).

`known_networks.json`
networks the machine joined (`NetworkList`): name, kind, category, gateway hardware address and DNS suffix of each signature, first and last connection. Windows writes these two dates in local time: WAC gives them in UTC with the offset in force at their date (measured against the `NetworkProfile` log). They tell where the machine has been.

`program_usage.json`
what each user launched or used, source by source (`Source`): `FeatureUsage` (launches, switches, taskbar clicks, per application, `Count`), `JumplistData` (last use, `LastUsed`), the Program Compatibility Assistant's store (programs run), compatibility layers (`Layers`), the programs RADAR diagnosed, the taskbar's pinned items (`Taskband`), each decoded as a whole shortcut, and the programs chosen to open each file type (`FileExts`: `OpenWithList` with `Extension` and `MruPosition`, 0 for the most recent; `UserChoice`, the default program). `OpenWithProgids` is not collected: installers fill it, the user has no part in it.

`devices.json`
devices the machine knew (`Source`): USB and SCSI (`Enum`), with their values and their install, first install, last arrival and last removal dates (`Install`, `FirstInstall`, `LastArrival`, `LastRemoval`, UTC); portable devices (a phone, a camera); volume labels Windows Search kept per drive letter (`VolumeInfoCache`); disk, volume and portable-device interfaces (`DeviceClasses`), written when they first appeared.

`network_shares.json`
folders the machine shares (`LanmanServer\Shares`): name, path, settings as stored, and the decoded security descriptor — owner, and for each access control entry its type, mask, SID and name. The administrative shares (`C$`, `ADMIN$`) are not there: Windows does not store them.

`network_configuration.json`
network configuration read in the hives: host name and domain (`Global`), and for each adapter its description and driver, a forced hardware address (`NetworkAddress`), its current and permanent hardware address, its last IPv4 and IPv6 settings, and its DHCP lease (obtained and end, UTC). It stays readable once the machine is unplugged or the lease expired, which the live state no longer shows.

`event_log_configuration.json`
configuration of every event log read in the hives: the channels (`WINEVT\Channels`: `Enabled`,

`Type`, `OwningPublisher`, access rights `ChannelAccess`, size and retention when set) and the classic logs (`Services\EventLog`: file, `MaxSize`, `Retention`, access) and the ETW sessions started at boot (`WMI\Autologger`: `Start`, file, size). An intruder who disables a channel, shrinks a log, opens its access or stops a session such as `EventLog-Security` leaves the trace there; `KeyLastWrite` dates the last change of each.

`fat_volumes.json`

FAT12, FAT16, FAT32 and exFAT volumes, the suspect's USB keys and cards included (the collection's volume excepted): every entry, present or **deleted** (`Deleted`), its path, 8.3 name, size, attributes and dates. A name deleted on FAT keeps its long name unless overwritten; otherwise its 8.3 name, first letter lost (`NameIncomplete`). Each date is given in UTC and in local time derived from UTC, as in every artefact: one instant reads the same everywhere. FAT stores the local time Windows computed when it wrote it: a modification earlier than the creation is a date kept by a copy, written at the copy, and converted with the offset of that moment (`ModifiedWrittenAt Copy`, `ModifiedOffsetApplied`, `ModifiedAsStored`) — otherwise a winter file copied in summer would be an hour off. exFAT keeps its offset (`ModifiedOffset`…), the UTC date is exact. The collection copies only the volume's metadata (boot sector, FAT, directories), not the files' content; with `--binary`, a program of these volumes cited by an artefact (run from a key, for instance) is fingerprinted and collected as on NTFS.

`srum.json`

resource usage (SRUM, `sru\SRUDB.dat`), one entry per row of every table: bytes sent and received per application and per interface (an exfiltration shows there, with the program), CPU cycles, bytes read and written, connections; 30 to 60 days. `Table` (the provider's GUID), `Provider` (documented name: `NetworkUsage`, `ApplicationResourceUsage`…), then the columns by name (letters and digits only), and `App`, `UserSID`, `UserName` resolved through `SruDbIdMapTable`. The database is copied live: the last hour is in its logs `SRU*.log`, copied into the exhibit store for a replay (`esentutl /r`) on a copy.

`bits_jobs.json`

BITS jobs (`qmgr.db`): name, description, type, state, owner, **notification command line** (run when the transfer ends or fails: a persistence), delays, dates, and each file: source URL, destination, temporary file, sizes. The whole blob is given in hexadecimal: BITS 10 keeps more there (the creating program) at places not described.

`wmi_persistence.json`

WMI event subscriptions, read **offline** in the CIM repository copied raw (`wbem\Repository`): no WMI query, nothing run on the machine. In *every* namespace, each filter (WQL query), consumer (command line, VBScript or JScript script, log, mail, and any class an intruder defined), binding and timer, with all its properties (`Properties`: name and value; `CreatorSID` in text). A persistence invisible in the registry. The two dates at the start of an instance are not emitted: they are not creation dates.

`web_client.json`

each user's browsing in the Chromium browsers (Edge, Chrome, Brave, Vivaldi, Chromium, Opera), read in the `History` database of each browser profile. One record per URL (`Type Url`: title, visit and typed counts, last visit), per visit (`Visit`: instant, `Transition` — `typed`, `link`, `reload`… — and its qualifiers, the visit it came from `FromUrl`, duration), per download (`Download`: whole redirection chain `UrlChain`, page and tab of origin,

destination path, bytes received, state, opened, raw `DangerType`, SHA-256 when the browser computed it) and per term searched in the address bar (`SearchTerm`). Every record carries `Browser`, `BrowserProfile`, `SID` and `Source`. Chromium's times (microseconds since 1601, UTC) are converted exactly. The databases are read on a private copy with their journal, replayed as the browser would. Beside `History`, the exhibit store receives raw the profile's other databases (`Login Data`, `Web Data`, cookies, shortcuts, bookmarks, preferences) and `Local State`, whose DPAPI-protected key encrypts the saved passwords; never the caches. Firefox (`Browser Firefox`, each folder of `Profiles`) gives the same records from `places.sqlite` — visits with their type (`typed`, `link`, `download`…), downloads with their destination, end and size —, plus `TypedInput` (what was typed in the address bar and the page reached) and, from `formhistory.sqlite`, `FormEntry`: each value typed in a form or the search bar (`FieldName searchbar-history`), with its dates. Copied raw beside them: cookies, `logins.json` and `key4.db` (saved logins, encrypted), extensions, preferences, session. Finally `TypedUrl`: each address typed in Internet Explorer or File Explorer's address bar (`TypedURLs` of the `NTUSER.DAT`, `MruPosition` 0 for the most recent) with the instant it was typed (`TypedURLsTime`, `TypedUtc`). `SavedLogin`: each saved login (Chromium's `Login Data`, Firefox's `logins.json`) — site, account (in clear in Chromium, encrypted and so absent in Firefox), creation, last use, use count, refusal to save (`NeverSave`); `Cookie`: site, name, path, dates, flags. The passwords and the cookies' values, encrypted, are never decoded: the databases are in the exhibit store for an analysis that would hold the user's DPAPI key. And `CacheEntry` (`Browser WebCache`): each entry of the `WebCacheV01.dat` database of Internet Explorer, Edge Legacy and the WinINet applications, by container (`Container`: `History`, `MSHist…`, `Content`, `iedownload`, `BackgroundTransferApi`) — URL, account and page of a history entry (`User`, `Page`), access count, dates, cache file (`CachedFile`: its path, through the subfolder the entry names). Internet Explorer's engine (WinINet) still serves `mshta.exe` and scripts even when IE is redirected to Edge: each profile's cache (`INetCache\IE`) is collected, limited to the types an investigation needs — HTA, scripts, executables, archives, documents with macros, PDF —, and the HTA an attack had run is found there with its entry. A download of Internet Explorer 10 and 11 (the `iedownload` container) also gives `Download`: its URL, MIME type, temporary copy in the cache, the path it was saved to (`TargetPath`, or only `SuggestedName` when it was not), its size and the server's address — a format Microsoft does not document, established on a real download by IE 11 in the test machine. On Windows 7 and earlier, Internet Explorer 5 to 9 kept the same data in `index.dat` files (cache, history and its `MSHist01` folders, cookies, downloads): they are collected and converted into `CacheEntry` (`Browser Internet Explorer`), recovered (`Recovered`) and dropped (`RecordKind LEAK`) entries included; a weekly history's visit time, local, is converted, the expiry and last check, FAT dates without a time zone, are given as written. The modification time of the `MSHist` containers, which Windows writes in local time, is converted to UTC with the offset in force at its date. The database is copied live with its logs: its last entries may be only there (`esentutl /r` replay by the analyst).

`dpapi_master_keys.json`

DPAPI master keys of the system (`System32\Microsoft\Protect`, `Scope System`) and of each profile (`AppData\Roaming\Microsoft\Protect`, `User`). The files are copied as they are into the exhibit store, still encrypted by the user's password or the system's DPAPI secret, with `Preferred` and `CREDHIST`: with that password, the analysis can later decrypt the saved secrets (browser passwords, Wi-Fi, credential manager). The JSON gives their metadata

only, never the key: owner (`SID`), `Guid` (the header's, checked equal to the file's name), version, policy, size of each block — a non-zero `DomainKeyLength`: a domain account —, the file's dates, and `Preferred` with `PreferredExpiresUtc` for the key in use.

`startup_folders.json`
content of the startup folders, of all users (`Scope AllUsers`) and of each profile (`User`, with `SID`): every file, its dates and fingerprints; a shortcut is decoded (`Shortcut`, as in `recentdocs.json`).

`shortcuts.json`
shortcuts put there to be clicked, outside the Recent folder: Start menu of all users and of each profile, desktop of each profile and public desktop, Quick Launch and items pinned to the task bar (`Location`: `StartMenu`, `Desktop`, `QuickLaunch`, `TaskBar`; `Scope` and `SID` as above). Each shortcut is decoded as in `recentdocs.json`: target, arguments, volume and machine of origin. An uninstalled program leaves its shortcut there; an intruder plants a command line there, or retargets an existing shortcut at their binary while keeping the icon. The Start menu's Startup folder is not in it: it belongs to `startup_folders.json`. A desktop redirected elsewhere on a local disk (OneDrive, another volume, see `user_shell_folders.json`) is read there too.

`user_shell_folders.json`
each user's folders (`UserShellFolders` of their `NTUSER.DAT`): name (`Name`, and `FolderName` for those a GUID names, as the Downloads), value as written (`Value`), resolved path (`Path`, absent for a network share) and last write of the key. OneDrive moves the desktop and the documents, a company redirects them to a share: that is where the user's files are to be looked for, and a redirection set quietly is itself a trace.

`powershell_history.json`
PSReadLine history of each profile and each host (`Host`: `ConsoleHost`, the ISE, VS Code): the commands in the order they were typed, a command on several lines joined. No date per command: `SourceModified` dates the last one.

`activities_cache.json`
Windows Timeline (`ActivitiesCache.db` of each account of each profile, with its `-wal` log): every row of the `Activity` and `ActivityOperation` tables (`Table`), all its columns. On Windows 10, the applications opened, the files and pages viewed (`Payload`: title, URI), the time spent in the foreground (`Payload` `UserEngaged`, from `StartTime` to `EndTime`) and, when clipboard history is on, what was copied (`ClipboardPayload`); on Windows 11, which dropped the Timeline, only the synchronised settings. The dates are Unix times in UTC, given in UTC and in local time; the GUIDs in .NET's byte order; a zero date or GUID is not emitted. The codes (`ActivityType`, `ActivityStatus`, `OperationType`) are given as stored: Microsoft does not document them.

`error_reports.json`
Windows error reports (`Report.wer`) of the machine and of each profile: program, path, instant (`EventTimeUtc`), problem signature (faulting module, version), every value of the report in `Values`.

`defender_log.json`
Microsoft Defender's support logs (`MPLog-*.log`): every line as written, dated when it starts with a UTC date. Run-time scans, detections, exclusions, updates. The WPP traces of the

same folder (`MpWpp*.bin`, binary ETW traces only Microsoft's private symbols decode) are copied raw into the exhibit store, not converted.

`defender_detections.json`
one entry per DetectionHistory file, which survives the clearing of the event log: threat, resource (file, key), account, process, initial detection date, quarantine entry, and the named `ThreatTracking` properties (fingerprints, size, start time). The format is undocumented: only the fields whose position was confirmed against `MSFT_MpThreatDetection` are named; the others are given as read (`UnnamedFields`: offset, type, value), without a meaning made up.

`defender_quarantine.json`
quarantine entries: threat, instant of the quarantine, original path of the removed file, its creation and modification dates, attributes, size, and the fingerprint of its content (`ContentSha256`). The quarantine is copied **encrypted**, as Defender keeps it: nothing malicious is written in clear, on the collection medium or on the analysis workstation. The content is decrypted in memory only, for its fingerprint.

`hosts.json`
entries of the `hosts` file: address and names, line number; an untouched Windows has none.

6.5.3 Files and browsing

`mft.json`
every file and directory of every fixed NTFS volume (`Volume`: `C:`, `G:`…; a record number only means something within its volume), present or **deleted** (`Deleted`) as long as NTFS has not reused its record: rebuilt path, size, attributes, and two sets of four UTC dates — `CreatedUtc`… (the `$STANDARD_INFORMATION` attribute, what Explorer shows and what a timestomping tool rewrites) and `FileNameCreatedUtc`… (the `$FILE_NAME` attribute, written by NTFS alone). A `$STANDARD_INFORMATION` date earlier than the `$FILE_NAME` one betrays timestomping. `PathIncomplete` marks a path one of whose parent directories was reused since: the path is not made up. A file with hard links (`HardLinks`) is one record under several names — `System32\crypt32.dll` and its original under `WinSxS`: `HardLinkPaths` gives its other paths, each one searchable. `SecurityId` is the number of its security descriptor in `security_descriptors.json`.

`security_descriptors.json`
security descriptors of every fixed NTFS volume (`$Secure:$SDS`): each once, however many files share it, identified by `Volume` and `SecurityId` — the key every file of `mft.json` carries. It gives the **owner** (`Owner`, `OwnerName`), the group, and each entry of the DACL and SACL (type, flags, access mask, SID and name). An intruder's tool dropped by their account, a system file no longer owned by TrustedInstaller, an ACL opened to Everyone are read there, deleted files included. NTFS stores with each descriptor a hash of its bytes: WAC recomputes it, and `HashMismatch` flags a descriptor that does not match it.

`usn_journal.json`
the change journal of every fixed NTFS volume (`$UsnJrnl:$J`, `Volume`; the `Usn` numbers rise within a volume): every creation, write, rename and **deletion**, dated, with its reasons

(`Reasons`, an unknown bit kept in hexadecimal) and the path rebuilt through the `$MFT`. It keeps the trace of files dropped then erased, which the `$MFT` has already forgotten. The `$MFT`, the change journal, `$LogFile` (the transaction journal, not converted) and `$TxfLog` (the journal of transactional NTFS, a CLFS log, its containers and the `$Tops:$T` stream, not converted) are also copied raw into the exhibit store, to rebuild the timeline later with another tool.

`ntfs_volumes.json`
every fixed NTFS volume, read in its boot sector (`$Boot`, recognised by its signatures before any field): serial number (`VolumeSerial`, the 32 bits shortcuts, jump lists and Prefetch files record of their target's volume, and `VolumeSerial64` whole), sector, cluster, record and index block sizes, place of the `$MFT` and of its mirror. Then the change journal's header (`$UsnJrnl:$Max`, `UsnJournal`): maximum size, allocation delta, `UsnJournalId`, which is the journal's **creation** date (`Created`), and the first number still present. A journal deleted and recreated (`fsutil usn deletejournal`) shows a creation far later than the installation.

`recentdocs.json`
recent-document shortcuts: target, timestamps of the shortcut and of the target, target size, volume (type, serial number, label), network path, shell item list, and the extra data blocks: the tracker block (`Tracker`: NetBIOS name of the machine the target was on, tracking identifiers, and what their version-1 GUID reveals — when the tracking began and the machine's hardware address), paths with environment variables, known folder, target properties.

`jumplistAutomaticDestinations.json`, `jumplistCustomDestinations.json`
per-application jump lists, with the shortcuts they hold; in the automatic ones, each shortcut comes with its DestList entry (last access, machine name, tracking identifiers with their date and hardware address, pin status, number of openings `InteractionCount`, rank `MruPosition` — 0 for the most recent —, name of the known folder targeted).

`shellbags.json`
folders browsed in Explorer, as a rebuilt tree; for each folder, its rank among its siblings (`MruPosition`, 0 for the most recent), the write time of the key holding it (`LastWriteTime`) and of its own key (`KeyLastWriteTime`: its addition, or the browsing of a subfolder), and its `NodeSlot`. The "first" and "last interaction" of other tools are only readings of these two times.

`mrus.json`, `mruApps.json`
most-recently-used files and applications.

`zone_identifiers.json`
`Zone.Identifier` streams ("Mark of the Web") that browsers and mail clients write on what they download: file, zone (`ZoneId`, `ZoneName` — `Internet` for 3), page of origin (`ReferrerUrl`) and the file's address (`HostUrl`), every value in `Values`. Read in the copy of the `$MFT`, where this small stream is resident: no further read of the examined machine; a non-resident stream, rare, is flagged (`ContentNotResident`).

`recycle_bin.json`
files sent to the recycle bin, on every fixed NTFS volume, from their `$I` index: original path, size, instant of the deletion (`DeletedUtc`), and the user (`SID`). Only the `$I` are collected; the files themselves (`$R`) are in `mft.json`.

6.5.4 Devices

`setupapi.json`
sections of `setupapi.dev.log` and its rotations: operation (`Operation`), device or driver (`Subject`, with a USB key's serial number), start and end converted from the examined machine's local time, outcome. The first connection of a device is dated there, even when the `USBSTOR` key was cleaned.

`Usbstor.json`
USB storage devices known to the system: `Vendor`, `Product`, `Revision` and `Type`, from `DeviceId`; `SerialNumber`, from `InstanceId` — absent when Windows made the identifier up (second character "&"), the device declaring none; first and last insertion dates.

`mounted_device.json`
mapping between drive letters and devices. `Drive` names the letter or the volume; `Type` gives the form of the value: `GPT partition` (`PartitionGuid`), `MBR partition` (`DiskSignature` and `PartitionOffset`, in bytes), `Device path` (`Device`, the device path, with the serial number of a USB key) or `Unknown` (`Data`, the bytes in hexadecimal).

6.5.5 Event logs (`--events`)

`events.json` holds one object per event:

`EvtSystem*`
the system section: provider, identifier, level, task, keywords, creation time, record number, process and thread, channel, computer, user;

`EvtEventData`
the event's data, as a list of `Name` / `Value` pairs;

`EvtEventMessage`
the readable message, as Event Viewer would show it, rebuilt from the provider's resources collected on the machine;

`EvtSourceLog`
the log file the event comes from. One channel can be carried by several files (current log and archives) whose record numbers overlap;

`EvtRecovered`
`true` for an event of a *cleared* log: Windows resets the chunk header without wiping the records, which WAC reads again. Event Viewer no longer shows them; they remain the trace of what the clearing meant to erase, and must never be read as live events.

> **Incomplete messages**
> A `%1`, `%2`… mark left in a message means the event does not carry the expected data: it is kept visible so that the sentence does not look complete. Not every provider ships message resources: their events then have no `EvtEventMessage`, but their data are complete.

CHAPTER 7

Reading the data

7.1 Timestamps

Every date is emitted twice: `X` in local time and `XUtc` in UTC. The local time is the **suspect's**, computed with the time zone read from its `SYSTEM` hive — not the collecting machine's. Dates carry the fraction of a second when the source provides it, and only to that source's precision: seven digits for a `FILETIME`, three for a date to the millisecond, none for a date to the second (FAT date, precise to two seconds, text date, Unix time).

Every date carries the offset in force **at that date**, daylight saving time included: a January date collected in summer in Paris is at `+01:00`, a July date at `+02:00`. `WAC` applies the rules of the examined machine year by year (`Dynamic DST` key of the `SOFTWARE` hive): a 2005 date in the United States follows the rules of 2005. The local time written is always derived from the UTC time: one instant reads the same in every artefact. Two dates have no knowable time zone and are given **as written**, without offset nor UTC, with `TimeZone` saying so: the date of an item seen inside an archive (`ModifiedAsWritten`; a ZIP entry keeps the clock time of the machine that made the archive) and an OLE date of a property store (`AsWritten`). Placing them on a timeline needs their origin: `WAC` does not make it up. For a value the artefact stores in local time (FAT volume, `setupapi.dev.log`), the hour repeated in autumn is read as its first occurrence, and the hour skipped in spring with the offset before the change, as Windows does.

7.2 Account names

The fields that name an account (`SIDName`, `Owner`, `RunAs`) come from the evidence alone, without asking the system or the network: local accounts and groups of the SAM database (in the language of the examined machine), well-known SIDs under their canonical names (`SYSTEM`, `LOCAL SERVICE`…), service SIDs (`NT SERVICE\…`), and the profile folder for a domain account. A SID none of them names is published without a name: the SID remains the evidence, for the analyst to resolve against the domain.

The Default profile. `C:\Users\Default` is the template copied for every new profile: a Run value, a shortcut or an association put there reaches every account created afterwards. WAC reads it as a profile (`NTUSER.DAT` and `UsrClass.dat` hives, desktop, Start menu, Recent…). Belonging to no account, it has no SID: its entries carry the SID `S-0-0-0`, zeros only, in a SID's form so that tools link them, but no SID at all (revision 0), and `Profile`: `Default` instead of `SIDName`. It is not `S-1-0-0`, Windows' real NULL SID.

7.3 GUID names

The fields that name a GUID (COM class, known folder, handler of a task) come first from a reference table, then from the examined machine's hives: its classes (`SOFTWARE\Classes\CLSID`), its known folders (`FolderDescriptions`), and each user's classes (`UsrClass.dat`). A component installed on the machine is therefore named, with the module that serves it. A GUID nothing names is published without a name.

7.4 A missing field is not an empty field

A value with no source is not emitted at all. An empty key would read as a failed read, and could not be told apart from "the hive does not hold this value". What could not be read is recorded in `investigation.json`.

7.5 Artefact not collected

When a whole artefact could not be read, its file says so explicitly (the text is produced in French):

```
{
  "Artifact": "USBSTOR (registre)",
  "CollectionStatus": "NotCollected",
  "Error": "0x02 Le fichier spécifié est introuvable.",
  "Note": "La lecture de cet artefact a échoué : l'absence de données
           ci-dessus ne signifie PAS qu'aucune trace n'existe sur le système."
}
```

> **Warning**
> An empty artefact file and a "NotCollected" file do not say the same thing. The first means no entry was found; the second, that the source could not be read.

7.6 Undecoded data

An item WAC cannot decode (unknown shell item, unrecognised extension block) comes with its raw bytes in hexadecimal, so that an analyst can decode it later. With `--dump`, the raw value of shellbags and shortcuts is always added.

7.7 Paths

Paths and values are delivered **as they are on the disk**, untransformed: escaping happens only when the JSON is written.

CHAPTER 8

Traces left on the examined machine

Read this before using WAC on a case. This chapter lists what WAC does to the machine, operation by operation, including what it cannot avoid.

8.1 What WAC still does

Operation	Trace
Direct volume reading	One access per volume actually read (usually one). No file opened, so no access date modified. If object-access auditing is enabled, opening the volume may be logged (Security 4656/4663).
Writing the results	On the collection medium only. Nothing is written to the examined disk.
Replaying hive logs	On the working copy only, with an undo journal.
Service state	One read-only query to the Service Control Manager. No service is opened or modified.
Processes	A kernel snapshot; no process is opened. Owners: one query to the Terminal Services service.
Sessions	One read-only query to LSASS.
Network state	Reads of the TCP/IP stack's tables in memory: no packet sent, no name resolved. `iphlpapi.dll` is loaded only when `artefacts.network` is on.
Boot time	One in-memory query to the kernel.
Event logs (`--events`)	Read through the same direct volume access: the logging service is not solicited.
Referenced files (`--binary`)	Read through the same direct access. No file opened.
Authenticity check (`--binary`)	Catalogs read through the same direct access; verification in memory. No service solicited, no access to the certificate store, no network access.

8.2 What WAC no longer does

Earlier versions created a Volume Shadow Copy, went through COM and WMI, queried the Task Scheduler and the logging service, opened every process and every service, read the profile list in the live registry and opened every referenced file to fingerprint it. All these operations, which left entries in the logs or changed access dates, have been removed.

8.3 What no tool can avoid

These traces are listed so that the analyst recognises WAC's footprint instead of taking it for the activity under study:

- **running `WAC.exe` is an execution**: it produces a Prefetch file for `WAC.exe`, an Amcache entry and, if process-creation auditing is on, a Security 4688 event. Expect to find WAC in the very artefacts it collects;
- **plugging in the USB stick** registers it in `USBSTOR`, `MountedDevices` and the related logs;
- **logging on** to run the tool creates a session and its log entries;
- WAC **takes up memory** during the collection, which may push pages out to the page file. Its reading of the volume bypasses the cache (unbuffered reads): the gigabytes read do not replace the machine's file cache.

CHAPTER 9

Known limitations

Volumes neither NTFS nor FAT
A file on a CD-ROM is not read: it is reported as unread.

Per-user variables
A path written with `%APPDATA%`, `%LOCALAPPDATA%` or `%USERPROFILE%` (in a scheduled task, for instance) is not resolved: the account concerned is not known there, and guessing would fingerprint another file.

Deleted executable
A process whose executable has been deleted from the disk has no fingerprint: only memory is left, outside WAC's scope.

Event messages
About a third of the events get a readable message on the test machine; the other providers ship no usable resources. The data themselves are always complete.

Signed but not authenticated
A few kinds of genuinely signed files are still collected, the cautious side: Microsoft Store applications (signed at package level, not file by file) and Windows Script Host scripts signed only within their text.

CHAPTER 10

Troubleshooting

Message or finding	Cause and remedy
“Check administrator rights” fails	The prompt was not started as administrator. Start it again with administrator rights.
“Check OS >= Windows 10” fails	System older than Windows 10: not supported.
“Checking the collection medium” fails, directory not empty	The output folder holds an earlier collection. Give a new folder with `--output`.
“Checking the collection medium” fails, not enough space	The medium is too small for the chosen options (section 3.2). Use a larger medium or drop `--binary`.
An artefact file holds `"CollectionStatus": "NotCollected"`	The source could not be read; the reason is in `Error`. The other artefacts are not affected.
Failed exhibits in the manifest	Referenced files missing from the disk (uninstalled software, service profile without a hive), non-NTFS volume… The reason is in `Result`.
The log reports binaries “hashed without being copied”	Space ran out during collection: the fingerprints are complete, the copies are not.
Need to understand a read failure	Run again with `--loglevel=2` (and, for NTFS reading, `--debug`) and read `WAC.exe.log`.

APPENDIX A

Glossary

Artefact
Trace left by the activity of the system or of a user, usable in an investigation.

Exhibit store (`exhibits`)
Directory of the raw copies, sealed, never rewritten.

Fingerprint
Value computed on a file's content (MD5, SHA-1, SHA-256): different contents have different fingerprints.

Hive
File holding part of the registry (`SYSTEM`, `SOFTWARE`, `ntuser.dat`…).

Manifest
File identifying each exhibit of the exhibit store.

`$MFT`
File table of an NTFS volume.

Seal
The manifest's fingerprint, kept apart: it reveals any change to the manifest.

Transaction log
`.LOG1`/`.LOG2` file holding a hive's recent changes, not yet written into it.

Working copy (`working`)
Directory of the working copies, on which the analysis and any modification are made.

WOF / Compact OS
Compressed storage mode of the system files of Windows 10 and 11.